ESSENTIALS OF WEB PROGRAMMING
UNIT-II

1. Introduction to CSS and Types of Style Sheets

1.1 Introduction

In Unit-I, we learned that HTML is used to create the structure of a webpage by defining elements such as headings, paragraphs, tables, forms, and images. Although HTML organizes the content, it does not provide an attractive appearance. A webpage developed using only HTML appears plain and lacks visual appeal.

To improve the appearance of webpages, Cascading Style Sheets (CSS) is used. CSS controls the presentation of HTML elements by defining colors, fonts, spacing, borders, layouts, and many other visual properties. It separates the content from the design, making webpages more attractive, consistent, and easier to maintain.

Today, almost every modern website, such as Google, Amazon, Flipkart, or a college website, uses CSS to create a professional and user-friendly interface.

1.2 Need for CSS

When HTML was first introduced, it was mainly designed to organize and display content rather than design webpages. As websites became more interactive and visually appealing, developers required a separate technology to control the appearance of webpages.

CSS was introduced to solve this problem by allowing developers to define styles separately from HTML content. Instead of applying the same formatting repeatedly to multiple HTML elements, CSS enables a single style rule to be reused throughout a website. This reduces code duplication, improves consistency, and simplifies website maintenance.

Scenario-Based Example

A university website contains 100 webpages. The university decides to change the heading color from blue to green.

Without CSS, the developer must modify every heading on all 100 webpages individually.

With CSS, the developer changes the color in one stylesheet, and the update is automatically reflected across all webpages.

This demonstrates why CSS is an essential technology for modern web development.

1.3 Cascading Style Sheets (CSS)

Definition: Cascading Style Sheets (CSS) is a style sheet language used to describe the presentation and appearance of HTML documents. It controls the layout, colors, fonts, spacing, borders, and positioning of webpage elements.

CSS works together with HTML. While HTML defines what should appear on a webpage, CSS defines how it should appear.

For example,

HTML creates a heading.

CSS changes its color, font size, and alignment.

Example

```
<h1>ABC College of Engineering</h1>

h1{

    color: blue;

    text-align: center;

}
```

The browser displays the heading in blue color and aligns it at the center of the webpage.

1.4 Advantages of CSS

CSS provides several advantages in web development.

It separates webpage content from presentation, making HTML code simpler and easier to understand. A single stylesheet can be applied to multiple webpages, ensuring a consistent design throughout the website. CSS also reduces development time because style changes can be made in one place instead of modifying every webpage individually. In addition, CSS improves webpage loading speed, simplifies maintenance, and helps create responsive websites that work on different devices.

Scenario-Based Example

A hospital manages separate webpages for Doctors, Departments, Appointments, and Contact Information. By using a common CSS stylesheet, all webpages display the same fonts, colors, and layout, giving the website a uniform and professional appearance.

1.5 Types of Style Sheets

CSS can be applied to HTML documents in three different ways.

- Inline Style Sheet
- Internal Style Sheet
- External Style Sheet

Each method has its own purpose depending on the size and complexity of the website.

1.5.1 Inline Style Sheet

Definition: An Inline Style Sheet applies CSS directly to an individual HTML element using the style attribute.

Inline CSS affects only the element in which it is written. It is generally used when a unique style is required for a single element.

Syntax

<tag style="property:value;">

Example

<h1 style="color:blue;">ABC College of Engineering</h1>

In this example, the heading color is changed to blue using the style attribute.

Scenario-Based Example

A college website wants to display an urgent admission announcement in red while all other headings remain unchanged. Since only one heading requires special formatting, an inline style is appropriate.

1.5.2 Internal Style Sheet

Definition: An Internal Style Sheet defines CSS rules within the <style> element placed inside the <head> section of an HTML document.

The styles apply to all matching elements within that webpage.

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
h1{
color:blue;
text-align:center; }
</style>
</head>
<body>
<h1>ABC College</h1>
```

```
</body>

</html>
```

In this example, every <h1> element in the webpage appears in blue and is center aligned.

Scenario-Based Example

A department webpage contains several headings that should have the same color and alignment. Instead of repeating inline styles for every heading, an internal stylesheet is used.

1.5.3 External Style Sheet

Definition: An External Style Sheet stores CSS rules in a separate file with the .css extension and links it to one or more HTML documents.

This is the most commonly used method because it allows multiple webpages to share the same stylesheet.

Example

HTML File

```
<head>

<link rel="stylesheet" href="style.css">

</head>
```

style.css

```
h1{

color:blue;

font-family:Arial; }
```

The browser reads the CSS rules from style.css and applies them to the linked HTML page.

Scenario-Based Example

A university website contains separate webpages for Home, Departments, Admissions, Placements, and Contact. To maintain the same design across all webpages, the developer creates one external stylesheet and links it to every webpage.

1.6 Complete HTML Program

```
<!DOCTYPE html>
<html>
<head>
<title>Introduction to CSS</title>
<style>
h2{
color:green;
text-align:center;
}
</style>
<link rel="stylesheet" href="style.css">
</head>
<body>
<h1 style="color:blue;">ABC College of Engineering</h1>
<h2>Department of Computer Science</h2>
<p>Welcome to the college website.</p>
</body>
</html>
```

Program Explanation

The webpage demonstrates the three methods of applying CSS. The <h1> heading uses an inline style to display blue text. The <h2> heading is styled using an internal stylesheet placed within the <style> element. The <link> element connects an external stylesheet, allowing additional styles to be applied from a separate CSS file. This example helps students understand how CSS can be applied at different levels.

1.7 Scenario-Based Questions

Scenario 1

A college website wants to change the color of only one admission notice.

Question: Which type of stylesheet should be used?

Answer: Inline Style Sheet

Scenario 2

A department webpage contains several headings that should have the same style.

Question: Which type of stylesheet is suitable?

Answer: Internal Style Sheet

Scenario 3

A university website has more than 200 webpages, and all pages must follow the same design.

Question: Which type of stylesheet is the best choice?

Answer: External Style Sheet

Scenario 4

A company wants to update the font style of its entire website by modifying only one file.

Question: Which type of stylesheet should be used?

Answer: External Style Sheet

1.8 Important Questions

Question 1: What is CSS?

Answer: CSS (Cascading Style Sheets) is a style sheet language used to describe the presentation and appearance of HTML documents.

Question 2: Why is CSS required?

Answer: CSS improves the appearance of webpages, separates content from presentation, reduces code duplication, and simplifies website maintenance.

Question 3: Name the three types of CSS.

Answer: Inline Style Sheet, Internal Style Sheet, and External Style Sheet.

Question 4: Which type of stylesheet is most suitable for large websites?

Answer: External Style Sheet.

Question 5: Which HTML element is used to link an external stylesheet?

Answer: <link rel="stylesheet" href="style.css">

1.9 Summary

CSS is a style sheet language that enhances the appearance of HTML webpages by controlling colors, fonts, spacing, layouts, and other visual properties. It separates presentation from content, making webpages attractive, consistent, and easier to maintain. CSS can be applied using Inline, Internal, and External Style Sheets, each serving different development needs. Among these, the External Style Sheet is the preferred approach for professional web development because it provides consistency and simplifies maintenance across multiple webpages.

2. CSS Selectors, Pseudo Classes and Pseudo Elements

2.1 Introduction

A webpage usually contains multiple HTML elements such as headings, paragraphs, tables, images, and hyperlinks. In most cases, a developer does not want to apply the same style to every element. Instead, different styles are applied to specific elements depending on the design requirements. CSS provides selectors to identify HTML elements and apply styles only to the required elements. In addition, CSS offers pseudo-classes and pseudo-elements to style elements based on their state or a specific part of their content, making webpages more interactive and visually appealing.

2.2 CSS Selectors

Definition: A CSS selector is a pattern used to select one or more HTML elements on which CSS styles are applied.

Selectors help developers target specific elements without affecting the entire webpage. Depending on the requirement, CSS provides different types of selectors such as element selectors, class selectors, ID selectors, and universal selectors.

2.3 Element Selector

Definition: An element selector selects all HTML elements of a particular type and applies the specified style to them.

The name of the HTML element itself is used as the selector.

Syntax

```
element{
    property:value;
}
```

Example

```
<h1>ABC College</h1>

<h1>Department of Computer Science</h1>

h1{

        color:blue; }
```

Both headings appear in blue because the selector applies to every <h1> element.

Scenario-Based Example

A college website contains several headings. The developer wants all headings to appear in blue. Instead of styling each heading individually, an element selector is used.

2.4 Class Selector

Definition: A class selector selects all HTML elements that belong to a particular class.

A class is defined using the class attribute and is represented in CSS using a dot (.).

Syntax

```
.classname{

        property:value; }
```

Example

```
<p class="notice">Admissions Open</p>

<p class="notice">Last Date: 30 June</p>

.notice{

        color:red;

        font-weight:bold;
```

```
}
```

Only the paragraphs belonging to the notice class are displayed in red.

Scenario-Based Example

A university website displays important announcements in red. Instead of applying styles separately to each announcement, all announcements are assigned the same class.

2.5 ID Selector

Definition: An ID selector selects a unique HTML element using its id attribute.

Unlike a class, an ID should be unique within a webpage. In CSS, an ID selector is represented using the hash (#) symbol.

Syntax

```
#idname{
    property:value;
}
```

Example

```
<h1 id="title">ABC College of Engineering</h1>
#title{
    color:green;
    text-align:center;
}
```

Only the heading with the ID title is styled.

Scenario-Based Example

A college homepage contains one main heading that should be displayed differently from all other headings. An ID selector is used because the heading is unique.

2.6 Universal Selector

Definition: The universal selector selects all HTML elements present on a webpage.

It is represented using the asterisk (*) symbol.

Syntax

```
*{
    property: value;
}
```

Example

```
*{
    font-family: Arial;
}
```

All elements on the webpage use the Arial font.

Scenario-Based Example

A company wants every webpage to use the same font family. The universal selector applies the font to all HTML elements.

2.7 Pseudo Classes

Definition: A pseudo-class is used to define the special state of an HTML element.

Pseudo-classes allow styles to be applied when an element is in a particular condition, such as when the mouse pointer moves over a link or when a user clicks it.

The colon (:) symbol is used before the pseudo-class name.

Example

```
a:hover{
    color:red; }
```

When the mouse pointer moves over the hyperlink, its color changes to red.

Common Pseudo Classes

- :hover
- :visited
- :active
- :focus

Scenario-Based Example

An online shopping website changes the colour of the Buy Now button when the user places the mouse pointer over it. This improves user interaction and provides visual feedback.

2.8 Pseudo Elements

Definition: A pseudo-element is used to style a specific part of an HTML element.

Pseudo-elements allow developers to style only the first letter, the first line, or insert additional content before or after an element.

The double colon (::) notation is commonly used.

Example

```
p::first-letter{
        font-size:30px;
        color:blue;
}
```

The first letter of every paragraph appears larger and blue.

Common Pseudo Elements

- ::first-letter
- ::first-line
- ::before

- ::after

Scenario-Based Example

A news website displays the first letter of every article as a large decorative letter to make the article more attractive. This effect is achieved using the :: first-letter pseudo-element.

2.9 HTML Program

```
<!DOCTYPE html>
<html>
<head>
<title>CSS Selectors</title>
<style>
h1{
color:blue;
}
.notice{
color:red;
font-weight:bold;
}
#title{
text-align:center;
}
a:hover{
color:green;
}
p::first-letter{
```

```
font-size:28px;
color:blue;
}
</style>
</head>
<body>
<h1 id="title">ABC College of Engineering</h1>
<p class="notice">Admissions are open for 2026.</p>
<p>Welcome to the Department of Computer Science.</p>
<a href="admission.html">Apply Online</a>
</body>
</html>
```

Program Explanation

The element selector changes the colour of all <h1> headings. The class selector styles the admission notice. The ID selector centers the main heading. The pseudo-class changes the hyperlink colour when the mouse pointer is placed over it, while the pseudo-element enlarges and colours the first letter of the paragraph.

2.10 Scenario-Based Questions

Scenario 1

A college website wants all headings to appear in blue.

Question: Which CSS selector should be used?

Answer: Element Selector.

Scenario 2

A university website displays multiple admission notices in red.

Question: Which CSS selector is most suitable?

Answer: Class Selector.

Scenario 3

A company wants to style only the company name displayed at the top of the homepage.

Question: Which CSS selector should be used?

Answer: ID Selector.

Scenario 4

A shopping website changes the colour of a hyperlink when the user moves the mouse pointer over it.

Question: Which CSS feature is used?

Answer: Pseudo-class (:hover).

Scenario 5

A digital magazine displays the first letter of each article in a larger font.

Question: Which CSS feature is suitable?

Answer: Pseudo-element (::first-letter).

2.11 Important Questions

Question 1: What is a CSS selector?

Answer: A CSS selector is a pattern used to select one or more HTML elements on which CSS styles are applied.

Question 2: Differentiate between a class selector and an ID selector.

Answer: A class selector styles multiple elements that share the same class, whereas an ID selector styles one unique element identified by its id.

Question 3: What is a pseudo-class?

Answer: A pseudo-class is used to apply styles when an HTML element is in a particular state, such as :hover or :focus.

Question 4: What is a pseudo-element?

Answer: A pseudo-element is used to style a specific part of an HTML element, such as the first letter or first line.

Question 5: Which selector applies styles to all elements on a webpage?

Answer: The Universal Selector (*).

2.12 Summary

CSS selectors enable developers to apply styles to specific HTML elements efficiently. Element, class, ID, and universal selectors help identify elements based on different criteria, making webpage design more organized and maintainable. Pseudo-classes enhance user interaction by styling elements based on their state, while pseudo-elements allow developers to style specific parts of an element. Together, these features make webpages more attractive, interactive, and easier to manage. In the next topic, we will study CSS Colors, Fonts, Lists, and Backgrounds, which control the visual appearance of webpage content.

3. CSS Colors, Fonts, Lists and Backgrounds

3.1 Introduction

The appearance of a webpage plays an important role in attracting users and improving readability. A well-designed webpage uses suitable colors, readable fonts, properly formatted lists, and attractive backgrounds to present information effectively. CSS provides various properties to control these visual aspects without changing the HTML content. By applying CSS styles, developers can create webpages that are attractive, consistent, and easy to use.

For example, a college website may display headings in blue, use a professional font for course details, present departments as a styled list, and apply a background image to the homepage. These visual effects are achieved using CSS properties.

3.2 CSS Colors

Definition: CSS colors are used to define the color of text, borders, and other HTML elements, improving the appearance and readability of a webpage.

Colors help highlight important information and create a visually attractive user interface. CSS allows colors to be specified using color names, hexadecimal values, RGB values, and HSL values. For beginners, color names are the easiest to understand.

Syntax

```
selector{
        color: colorname;
}
```

Example

```
<h1>ABC College of Engineering</h1>
h1 {
```

```
    color: blue;
}
```

The browser displays the heading in blue.

Scenario-Based Example

A university website wants to display all admission announcements in red to attract the attention of students. The developer applies the color property to the announcement text.

3.3 CSS Fonts

Definition: CSS fonts are used to control the appearance of text by specifying its font family, size, style, and weight.

Different fonts improve readability and give webpages a professional appearance. CSS provides several font properties that allow developers to customize the text displayed on a webpage.

The most commonly used font properties are:

- font-family
- font-size
- font-style
- font-weight

1. Font Family

The font-family property specifies the typeface used for displaying text.

Example

```
p{
    font-family: Arial;
}
```

2. Font Size

The font-size property specifies the size of the text.

Example

```
h1{
    font-size:36px;
}
```

3. Font Style

The font-style property changes the appearance of text, such as making it italic.

Example

```
p{
    font-style: italic;
}
```

Font Weight

The font-weight property specifies the thickness of the text.

Example

```
h2{
    font-weight:bold;
}
```

Scenario-Based Example

A hospital website displays the names of doctors in bold and uses a larger font size for department headings so that patients can easily identify important information.

3.4 Styling Lists Using CSS

Definition: CSS allows developers to customize the appearance of HTML lists by changing the list marker, position, and spacing.

Lists are commonly used to display menus, product features, departments, services, and instructions. CSS improves their appearance and readability.

The most commonly used property is list-style-type.

Example

```
<ul>
<li>Computer Science</li>
<li>Artificial Intelligence</li>
<li>Information Technology</li>
</ul>
ul{
      list-style-type:square;
}
```

The browser displays square bullets instead of the default circular bullets.

Other commonly used values are:

- disc
- circle
- square
- decimal
- lower-alpha
- upper-alpha

Scenario-Based Example

A college website lists various departments. To improve the appearance of the department list, square bullets are used instead of the default bullets.

3.5 CSS Backgrounds

Definition: CSS backgrounds are used to define the background color or background image of an HTML element or an entire webpage.

Backgrounds improve the visual appearance of webpages and help create attractive website designs.

The most commonly used background properties are:

- background-color
- background-image

1. Background Color

The background-color property changes the background color of an element.

Example

```
body{
    background-color:lightblue; }
```

The browser displays the webpage with a light blue background.

Scenario-Based Example

A school website uses a light blue background to create a pleasant reading environment for students.

2. Background Image

The background-image property displays an image as the background of an HTML element.

Example

```
body{
    background-image:url("college.jpg");
}
```

The browser displays the specified image as the webpage background.

Scenario-Based Example

A tourism website displays a scenic image as the homepage background to attract visitors.

3.6 Complete HTML Program

```
<!DOCTYPE html>
<html>
<head>
<title>CSS Colors and Fonts</title>
<style>
body{
        background-color:lightyellow;
}
h1{
        color:blue;
        font-family:Arial;
        font-size:36px;
}
p{
        font-size:18px;
}
ul{
        list-style-type:square;
}
</style>
</head>
<body>
<h1>ABC College of Engineering</h1>
<p>Admissions are open for the academic year 2026.</p>
```

```
<ul>
<li>Computer Science</li>
<li>Artificial Intelligence</li>
<li>Data Science</li>
</ul>
</body>
</html>
```

3.7 Program Explanation

The program demonstrates the use of several CSS properties to improve the appearance of a webpage. The background-color property changes the webpage background to light yellow. The heading is displayed in blue using the color property, while font-family and font-size improve its readability. The paragraph uses a larger font size for better visibility. Finally, the department list is displayed using square bullets through the list-style-type property.

3.8 Scenario-Based Questions

Scenario 1

A university wants all department headings to appear in blue.

Question: Which CSS property should be used?

Answer: color

Scenario 2

A hospital website wants all patient instructions to appear in the Arial font.

Question: Which CSS property should be used?

Answer: font-family

Scenario 3

A college website wants department names to be displayed with square bullets.

Question: Which CSS property should be used?

Answer: list-style-type

Scenario 4

A company wants the homepage to display a light gray background.

Question: Which CSS property should be used?

Answer: background-color

Scenario 5

A travel website wants to display a scenic image behind the homepage content.

Question: Which CSS property should be used?

Answer: background-image

3.9 Important Questions

Question 1: What is the purpose of the color property in CSS?

Answer: The color property specifies the text color of an HTML element.

Question 2: Which CSS property specifies the font type?

Answer: font-family

Question 3: Which CSS property changes the size of text?

Answer: font-size

Question 4: Which CSS property is used to change the bullet style of a list?

Answer: list-style-type

Question 5: Differentiate between background-color and background-image.

Answer: The background-color property applies a solid color to the background of an element, whereas the background-image property displays an image as the background.

3.10 Summary

CSS colors, fonts, lists, and backgrounds are essential styling features that improve the visual appearance and readability of webpages. The color property controls text color, font properties define the appearance of text, list properties enhance the presentation of ordered and unordered lists, and background properties add colors or images to webpage elements. Together, these properties help developers create attractive, consistent, and user-friendly websites. In the next topic, we will learn about the CSS Box Model and Conflict Resolution, which explain how HTML elements occupy space and how CSS resolves style conflicts.

4. CSS Box Model and Conflict Resolution

4.1 Introduction

Whenever an HTML element is displayed on a webpage, it occupies a certain amount of space. For example, a heading, paragraph, image, or button is not displayed randomly; each element is placed inside an invisible rectangular box. The browser uses this box to determine the size of the element and the space around it. This concept is known as the CSS Box Model.

In addition, a webpage may contain multiple CSS rules that apply to the same HTML element. When different rules specify different styles for the same property, the browser must decide which style to apply. This process is called Conflict Resolution or the CSS Cascade. Understanding these concepts helps developers create well-structured, consistent, and visually appealing webpages.

4.2 CSS Box Model

Definition: The CSS Box Model is a layout model that represents every HTML element as a rectangular box consisting of content, padding, border, and margin.

The browser calculates the total size of an HTML element using these four components. A clear understanding of the Box Model is essential for controlling spacing, alignment, and webpage layout.

4.3 Components of the CSS Box Model

The CSS Box Model consists of four components arranged from the inside to the outside.

1. Content

Definition: The content is the actual information displayed inside an HTML element, such as text, images, or other HTML elements.

It is the central part of the box where the browser displays the element's data.

Example

```
<p>Welcome to ABC College.</p>
```

Here, "Welcome to ABC College." is the content of the paragraph element.

Scenario-Based Example

A college website displays the admission notification inside a paragraph. The displayed message forms the content area of the box.

2. Padding

Definition: Padding is the space between the content and the border of an HTML element.

Padding creates internal spacing, making the content appear less crowded.

Example

```
p{
    padding:20px;
}
```

The browser leaves 20 pixels of space between the paragraph text and its border.

Scenario-Based Example

A hospital website displays patient instructions inside a bordered box. Padding is added so that the text does not touch the border, improving readability.

3. Border

Definition: A border is the line surrounding the content and padding of an HTML element.

Borders help distinguish one element from another and improve the visual appearance of webpages.

Example

```
p{
    border:2px solid black;
}
```

The browser draws a black border around the paragraph.

Scenario-Based Example

An online examination portal displays important instructions inside a bordered box to make them easily noticeable.

4. Margin

Definition: Margin is the space outside the border of an HTML element that separates it from other elements.

Margins prevent elements from appearing too close to each other.

Example

```
p{
    margin:25px;
}
```

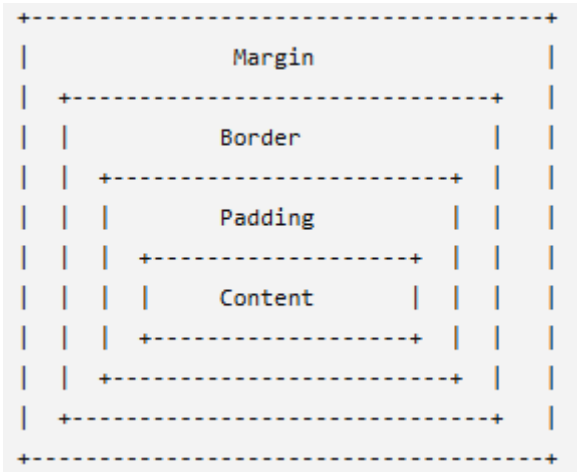
The browser leaves 25 pixels of space outside the paragraph border.

Scenario-Based Example

A university website displays department cards with equal spacing between them. Margins ensure that the cards do not overlap or appear crowded.

4.4 Illustration of the CSS Box Model

Every HTML element is displayed using the following structure:



The browser calculates the total size of an element by considering all four components.

4.5 HTML Program Demonstrating the Box Model

```
<!DOCTYPE html>
<html>
<head>
<title>CSS Box Model</title>
<style>
div{
        width:300px;
        padding:20px;
        border:2px solid blue;
        margin:30px;
        background-color:lightyellow;
}
</style>
```

```
</head>
<body>
<div>
<h2>ABC College of Engineering</h2>
<p>Admissions are open for the academic year 2026.</p>
</div>
</body>
</html>
```

4.6 Program Explanation

The element acts as a container for the heading and paragraph. The padding property creates space between the content and the border, improving readability. The border property draws a blue outline around the content, while the margin property leaves space between the element and surrounding webpage elements. The background-color property makes the container visually attractive.

4.7 CSS Conflict Resolution

Definition: CSS Conflict Resolution is the process by which the browser determines which CSS rule should be applied when multiple rules target the same HTML element.

Since a webpage may contain inline, internal, and external styles simultaneously, conflicts may occur. CSS follows specific rules, known as the Cascade, to decide which style has the highest priority.

4.8 Order of Priority in CSS

When multiple styles apply to the same element, the browser follows this priority:

1. Inline CSS (Highest Priority)
2. Internal CSS
3. External CSS
4. Browser Default Style (Lowest Priority)

The style with the highest priority overrides the others.

4.9 Example of Conflict Resolution

```
<!DOCTYPE html>
<html>
<head>
<style>
h1{
      color:green;
}
</style>
<link rel="stylesheet" href="style.css">
</head>
<body>
<h1 style="color:red;">ABC College</h1>
</body>
</html>
```

Suppose the external stylesheet contains:

```
h1{
      color:blue;
}
```

Explanation

Although three different styles define the color of the same heading, the browser applies the inline style because it has the highest priority.

4.10 Why is Conflict Resolution Important?

Large websites often use multiple CSS files and different styling techniques. Without conflict resolution rules, browsers would not know which style to apply. The CSS Cascade ensures that webpages display consistently and predictably.

Scenario-Based Example

A university website uses a common external stylesheet for all webpages. One department wants its homepage heading to appear in red instead of the default blue. The developer applies an inline style only to that heading. Because inline CSS has the highest priority, the department heading appears in red while the remaining pages continue to use the external stylesheet.

4.11 Scenario-Based Questions

Scenario 1

A college website wants to create space between the text and its border.

Question: Which CSS property should be used?

Answer: padding

Scenario 2

A shopping website wants to leave space between two product cards.

Question: Which CSS property is suitable?

Answer: margin

Scenario 3

A company website wants to display a blue line around an important announcement.

Question: Which CSS property should be used?

Answer: border

Scenario 4

A developer applies different colors using external, internal, and inline CSS to the same heading.

Question: Which style will the browser apply?

Answer: Inline CSS, because it has the highest priority.

Scenario 5

A webpage displays text very close to its border.

Question: Which CSS property should be increased?

Answer: padding

4.12 Important Questions

Question 1: What is the CSS Box Model?

Answer: The CSS Box Model is a layout model that represents every HTML element as a rectangular box consisting of content, padding, border, and margin.

Question 2: Name the four components of the CSS Box Model.

Answer: Content, Padding, Border, and Margin.

Question 3: What is the difference between padding and margin?

Answer: Padding is the space between the content and the border of an element, whereas margin is the space outside the border that separates the element from other elements.

Question 4: What is CSS Conflict Resolution?

Answer: CSS Conflict Resolution is the process of determining which CSS rule should be applied when multiple styles target the same HTML element.

Question 5: Arrange the following in order of priority:

Inline CSS, Internal CSS, External CSS.

Answer:

1. Inline CSS
2. Internal CSS
3. External CSS

4.13 Summary

The CSS Box Model is the foundation of webpage layout because every HTML element is displayed as a rectangular box consisting of content, padding, border, and margin. By adjusting these components, developers can control the spacing, size, and appearance of webpage elements. CSS also follows a Conflict Resolution (Cascade) mechanism to determine which style should be applied when multiple rules target the same element. Understanding these concepts enables developers to create organized, consistent, and professional web pages. In the next topic, we will study Introduction to JavaScript and the Need for JavaScript, where we will learn how JavaScript adds interactivity and dynamic behavior to webpages.

5. Introduction to JavaScript and Need for JavaScript

5.1 Introduction

In Unit-I, we learned that HTML is used to create the structure of a webpage, and in the previous topics of Unit-II, we studied CSS, which is used to improve the appearance of webpages. Although HTML and CSS can create attractive webpages, they cannot make webpages interactive. For example, HTML and CSS alone cannot validate a login form, calculate marks, display a welcome message, or respond to a button click.

To add intelligence and interactivity to webpages, JavaScript is used. JavaScript is a programming language that enables webpages to respond to user actions, process data, perform calculations, and update webpage content dynamically without reloading the page. Today, JavaScript is one of the most widely used programming languages for web development.

5.2 Need for JavaScript

Initially, webpages were static, meaning they could only display information. As the demand for interactive websites increased, developers required a technology that could respond to user actions such as clicking buttons, entering data, selecting options, or submitting forms.

JavaScript was introduced to overcome these limitations. It enables webpages to perform tasks such as validating user input, displaying messages, manipulating webpage content, and creating interactive features. As a result, websites become more responsive, user-friendly, and engaging.

Scenario-Based Example

A college admission portal asks students to enter their registration number. If the student clicks the Submit button without entering the registration number, the webpage should immediately display a warning message asking the student to complete the required field. This validation can be performed using JavaScript without sending the data to another webpage.

5.3 JavaScript

Definition: JavaScript is a high-level, lightweight, interpreted scripting language used to create interactive and dynamic webpages.

JavaScript works together with HTML and CSS to develop modern websites.

- HTML provides the structure of a webpage.
- CSS defines the appearance of the webpage.
- JavaScript adds functionality and interactivity to the webpage.

Together, these three technologies form the foundation of modern web development.

5.4 Features of JavaScript

JavaScript provides several features that make it suitable for developing interactive web applications.

It is an interpreted language, which means the browser executes JavaScript code directly without requiring compilation. JavaScript is lightweight and easy to learn, making it suitable for beginners. It supports event-driven programming, allowing webpages to respond to user actions such as mouse clicks and keyboard input. JavaScript also enables developers to manipulate HTML elements dynamically, validate forms, perform calculations, and update webpage content without refreshing the browser.

Scenario-Based Example

An online shopping website automatically updates the total bill whenever the customer changes the quantity of a product. JavaScript performs the calculation instantly without reloading the webpage.

5.5 Applications of JavaScript

JavaScript is widely used in modern web development to improve the functionality of websites.

Some common applications include:

- Form validation
- Online calculators
- Interactive menus
- Image sliders
- Digital clocks
- Online quizzes
- Shopping carts
- Login pages
- Popup messages
- Dynamic webpage updates

Scenario-Based Example

A banking website verifies whether the account number entered by the customer contains the required number of digits before allowing the form to be submitted. This validation improves user experience and reduces input errors.

5.6 Ways to Include JavaScript in HTML

JavaScript can be included in an HTML document in three different ways.

- Inline JavaScript
- Internal JavaScript
- External JavaScript

These methods are similar to the different ways CSS can be applied to an HTML document.

5.6.1 Inline JavaScript

Definition: Inline JavaScript is written directly inside an HTML element using an event attribute.

It is generally used for simple tasks involving a single HTML element.

Example

```
<button onclick="alert('Welcome to ABC College!')">
Click Here
</button>
```

When the user clicks the button, a message box displaying "Welcome to ABC College!" appears.

Scenario-Based Example

A college website displays a welcome message when a visitor clicks the Know More button.

5.6.2 Internal JavaScript

Definition: Internal JavaScript is written inside the <script> element within the same HTML document.

The <script> element is usually placed inside the <head> section or just before the closing </body> tag.

Example

```
<script>
function welcome(){
alert("Welcome Students"); }
</script>
```

Scenario-Based Example

A school website displays a greeting message whenever students click the Login button. The JavaScript function is written within the same HTML file.

5.6.3 External JavaScript

Definition: External JavaScript stores JavaScript code in a separate file having the .js extension and links it to one or more HTML documents. This method improves code reusability and simplifies website maintenance.

Example

HTML File

```
<script src="script.js"></script>
```

script.js

```
function welcome(){
alert("Welcome Students"); }
```

Scenario-Based Example

A university website contains multiple webpages such as Home, Admissions, Departments, and Placements. The same JavaScript file is linked to all webpages to perform common operations.

5.7 HTML Program

```
<!DOCTYPE html>
<html>
<head>
<title>Introduction to JavaScript</title>
<script>
function welcome(){
alert("Welcome to ABC College of Engineering"); }
</script>
```

```
</head>

<body>

<h1>ABC College of Engineering</h1>

<button onclick="welcome()"> Click Here </button>

</body>

</html>
```

5.8 Program Explanation

The webpage displays the college name and a button. The JavaScript function welcome() is written inside the <script> element. When the user clicks the Click Here button, the browser executes the function and displays a message box containing "Welcome to ABC College of Engineering." This example demonstrates how JavaScript adds interactivity to an HTML webpage.

5.9 Scenario-Based Questions

Scenario 1

A college website wants to display a welcome message when the user clicks a button.

Question: Which scripting language should be used?

Answer: JavaScript

Scenario 2

A banking website checks whether all required fields are filled before submitting the login form.

Question: Which technology performs this validation on the webpage?

Answer: JavaScript

Scenario 3

A shopping website automatically updates the total amount whenever the customer changes the product quantity.

Question: Which technology makes this possible?

Answer: JavaScript

Scenario 4

A university website wants to use the same JavaScript code on multiple webpages.

Question: Which method should be used?

Answer: External JavaScript

5.10 Important Questions

Question 1: What is JavaScript?

Answer: JavaScript is a high-level, lightweight, interpreted scripting language used to create interactive and dynamic webpages.

Question 2: Why is JavaScript required?

Answer: JavaScript is required to add interactivity, validate user input, perform calculations, manipulate webpage content, and respond to user actions.

Question 3: Name the three ways of including JavaScript in an HTML document.

Answer:

- Inline JavaScript
- Internal JavaScript
- External JavaScript

Question 4: Which HTML element is used to write JavaScript code?

Answer: <script>

Question 5: Which method is most suitable for large websites?

Answer: External JavaScript, because the same .js file can be shared across multiple webpages, making maintenance easier.

5.11 Summary

JavaScript is a powerful scripting language that transforms static webpages into interactive web applications. It works alongside HTML and CSS to provide functionality such as form validation, calculations, dynamic content updates, and user interaction. JavaScript can be included in an HTML document using inline, internal, or external methods, with external JavaScript being the preferred approach for larger websites. Understanding JavaScript is the first step toward developing modern, responsive, and interactive web applications.

6. Variables, Data Types and Operators

6.1 Introduction

In everyday life, we store different types of information such as a student's name, roll number, marks, age, and department. Similarly, a JavaScript program needs a way to store information so that it can be processed, displayed, or modified whenever required. This is achieved using variables.

The information stored in variables may be of different types, such as numbers, text, or logical values. These are known as data types. Once the data is stored, JavaScript uses operators to perform calculations, compare values, and combine information. Together, variables, data types, and operators form the foundation of JavaScript programming.

6.2 Variables

Definition: A variable is a named memory location used to store data that can be accessed and modified during the execution of a JavaScript program.

A variable act like a container that holds information. Instead of writing the actual value repeatedly, the programmer stores it in a variable and refers to the variable name whenever required.

Syntax

```
let variableName = value;
```

or

```
const variableName = value;
```

or

```
var variableName = value;
```

Note: In modern JavaScript, let and const are preferred over var because they provide better control over variables.

Example

```
let studentName = "Rahul";

let marks = 85;
```

In the above example,

- studentName stores the text Rahul.
- marks stores the numeric value 85.

Scenario-Based Example

A college admission portal stores the student's name and age after the user fills out the registration form.

```
let studentName = "Ananya";

let age = 18;
```

Instead of writing these values repeatedly, the program stores them in variables for future use.

6.3 Rules for Naming Variables

Variable names should follow certain rules so that JavaScript can identify them correctly.

A variable name must begin with a letter, an underscore (_), or a dollar sign ($). It cannot begin with a number. Variable names are case-sensitive, meaning studentName and StudentName are treated as different variables. Spaces are not allowed, and JavaScript keywords such as let, if, and while cannot be used as variable names.

1. Valid Variable Names

```
let studentName;

let totalMarks;
```

```
let _count;

let $price;
```

2. Invalid Variable Names

```
let 123name;

let student name;

let let;
```

6.4 Data Types

Definition: A data type specifies the kind of value stored in a variable.

Different types of information require different data types. JavaScript automatically identifies the type of data assigned to a variable.

The most commonly used data types for beginners are:

- Number
- String
- Boolean
- Undefined
- Null

1. Number

Definition: The Number data type stores both integer and decimal values.

Example

```
let marks = 95;

let percentage = 87.5;
```

Scenario-Based Example

A university stores the student's examination marks as numbers.

2. String

Definition: A String is a sequence of characters enclosed within single quotes (' '), double quotes (" "), or backticks ( ).

Example

```
let college = "ABC College";
```

Scenario-Based Example

A hospital stores the patient's name as a string because it contains alphabetic characters.

3. Boolean

Definition: A Boolean data type stores only one of two values: true or false.

Example

```
let isPassed = true;
```

Scenario-Based Example

An online examination system checks whether a student has passed the examination. If the student passes, the value is true; otherwise, it is false.

4. Undefined

Definition: A variable that is declared but not assigned any value has the value undefined.

Example

```
let department;
```

Since no value is assigned, JavaScript treats it as undefined.

5. Null

Definition: The null value represents the intentional absence of any value.

Example

```
let middleName = null;
```

Scenario-Based Example

A student registration form does not require a middle name. Therefore, the value can be assigned as null.

6.5 Operators

Definition: Operators are symbols used to perform operations on variables and values.

Operators help perform calculations, comparisons, and logical decisions in a JavaScript program.

1. Arithmetic Operators

Arithmetic operators perform mathematical calculations.

Example

```
let a = 20;
let b = 10;
console.log(a + b);
console.log(a - b);
console.log(a * b);
console.log(a / b);
console.log(a % b);
```

Operator	Purpose	Example
+	Addition	a + b

-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus	a % b

Scenario-Based Example

A college website calculates the total marks obtained by a student using the addition operator.

2. Assignment Operators

Assignment operators assign values to variables.

Example

let marks = 90;

Operator	Example
=	x = 10
+=	x += 5
-=	x -= 2

Scenario-Based Example

A banking application updates the account balance after depositing money using assignment operators.

3. Comparison Operators

Comparison operators compare two values and produce either true or false.

Example

```
let age = 18;

console.log(age >= 18);
```

Operator	Meaning
==	Equal to
!=	Not Equal to
>	Greater Than
<	Less Than
>=	Greater Than or Equal To
<=	Less Than or Equal To

Scenario-Based Example

An online voting portal checks whether the user's age is greater than or equal to 18 before allowing them to vote.

4. Logical Operators

Logical operators combine two or more conditions.

Example

```
let age = 20;

let citizen = true;

console.log(age >= 18 && citizen);
```

Operator	Meaning
&&	Logical AND
||	Logical OR
!	Logical NOT

Scenario-Based Example

A scholarship portal checks whether a student has both the required marks and valid documents before approving the scholarship.

6.6 Complete HTML Program

```
<!DOCTYPE html>
<html>
<head>
    <title>Variables and Operators</title>
</head>
<body>
<script>
let studentName = "Rahul";
let marks1 = 80;
let marks2 = 90;
let total = marks1 + marks2;
document.write("<h2>Student Name: " + studentName + "</h2>");
document.write("<h3>Total Marks: " + total + "</h3>");
</script>
</body>
</html>
```

6.7 Program Explanation

The program declares a variable named studentName to store the student's name and two variables, marks1 and marks2, to store examination marks. The addition operator (+) calculates the total marks, which is stored in the

total variable. Finally, the document.write() method displays the student's name and total marks on the webpage.

6.8 Scenario-Based Questions

Scenario 1

A college admission portal stores the student's name entered during registration.

Question: Which JavaScript concept is used to store the name?

Answer: Variable.

Scenario 2

A hospital management system stores a patient's age.

Question: Which data type is suitable?

Answer: Number.

Scenario 3

An online examination system checks whether a student has passed.

Question: Which data type is suitable for storing the result?

Answer: Boolean.

Scenario 4

A shopping website calculates the total price of products.

Question: Which type of operator is used?

Answer: Arithmetic Operator.

Scenario 5

A banking application verifies whether the account balance is greater than zero before allowing a withdrawal.

Question: Which type of operator is used?

Answer: Comparison Operator.

6.9 Important Questions

Question 1: What is a variable in JavaScript?

Answer: A variable is a named memory location used to store data that can be accessed and modified during program execution.

Question 2: What is a data type?

Answer: A data type specifies the kind of value stored in a variable.

Question 3: Name any four JavaScript data types.

Answer: Number, String, Boolean, and Undefined.

Question 4: What are operators?

Answer: Operators are symbols used to perform operations on variables and values.

Question 5: Name the four commonly used categories of operators.

Answer: Arithmetic, Assignment, Comparison, and Logical operators.

6.10 Summary

Variables, data types, and operators form the foundation of JavaScript programming. Variables store information, data types define the nature of that information, and operators perform calculations, comparisons, and logical operations. Understanding these concepts is essential before learning decision-making statements, loops, and functions. In the next topic, we will study Conditional Statements, Loops, Functions, and Arrow Functions, which enable JavaScript programs to make decisions, repeat tasks, and organize reusable code.

7. Conditional Statements, Loops, Functions and Arrow Functions

7.1 Introduction

A computer program does more than simply execute instructions in sequence. It often needs to make decisions, repeat tasks, and organize code into reusable blocks. For example, a college website checks whether a student is eligible for admission, an online examination system calculates marks for multiple students, and a banking application performs the same transaction repeatedly for different customers. JavaScript provides conditional statements, loops, and functions to perform these tasks efficiently. In modern JavaScript, arrow functions offer a shorter and more readable way of writing functions.

7.2 Conditional Statements

Definition: A conditional statement is a decision-making statement that executes different blocks of code based on whether a specified condition is true or false.

Conditional statements help programs make decisions instead of executing every statement. They are commonly used for checking age, marks, passwords, login details, and eligibility.

Types of Conditional Statements

JavaScript provides the following conditional statements:

- if
- if...else
- else if
- switch

1. The if Statement

Definition: The if statement executes a block of code only when the specified condition is true.

Syntax

```
if(condition)
{
    // statements
}
```

Example

```
let age = 20;
if(age >= 18)
{  document.write("Eligible to Vote"); }
```

If the condition is true, the message is displayed.

Scenario-Based Example

A university checks whether a student has paid the examination fee. If the payment is completed, the student is allowed to download the hall ticket.

2. The if...else Statement

Definition: The if...else statement executes one block of code when the condition is true and another block when the condition is false.

Syntax

```
if(condition)
{
    // True block
}
else
```

```
{
    // False block
}
```

Example

```
let marks = 45;

if(marks >= 50)
{
    document.write("Pass");
}
else
{
    document.write("Fail");
}
```

Scenario-Based Example

An online examination system displays Pass if the student's marks are 50 or above; otherwise, it displays Fail.

3. The else if Statement

Definition: The else if statement is used to test multiple conditions one after another.

Example

```
let marks = 82;

if(marks >= 90)
{
    document.write("Grade A+");
```

```
}
else if(marks >= 75)
{
      document.write("Grade A");
}
else
{
      document.write("Grade B");
}
```

Scenario-Based Example

A university assigns grades based on the marks obtained by students.

4. The switch Statement

Definition: The switch statement selects one block of code from multiple alternatives based on the value of an expression.

Syntax

```
switch(expression)
{
      case value1:
            statements;
            break;

      case value2:
            statements;
            break;
```

```
    default:
        statements;
}
```

Example

```
let day = 2;
switch(day)
{
    case 1:
        document.write("Monday");
        break;
    case 2:
        document.write("Tuesday");
        break;
    default:
        document.write("Invalid Day");
}
```

Scenario-Based Example

A railway reservation system displays the day of travel based on the number selected by the user.

7.3 Loops

Definition: A loop is a control structure that repeatedly executes a block of code until a specified condition becomes false.

Loops reduce code repetition and simplify programming.

1. The for Loop

Definition: The for loop is used when the number of repetitions is known in advance.

Syntax

```
for(initialization; condition; increment)
{
    // statements
}
```

Example

```
for(let i = 1; i <= 5; i++)
{
    document.write(i + "<br>");
}
```

Scenario-Based Example

A college website displays the roll numbers of five students.

2. The while Loop

Definition: The while loop repeatedly executes a block of code as long as the specified condition remains true.

Example

```
let i = 1;
while(i <= 5)
{
    document.write(i + "<br>");
    i++;
}
```

Scenario-Based Example

A ticket booking system keeps displaying available seats until all seats are filled.

3. The do...while Loop

Definition: The do...while loop executes the statements at least once before checking the condition.

Example

```
let i = 1;

do

{

      document.write(i + "<br>");

      i++;

}

while(i <= 5);
```

Scenario-Based Example

An ATM displays the main menu at least once before asking the customer whether to perform another transaction.

7.4 Functions

Definition: A function is a reusable block of code designed to perform a specific task whenever it is called.

Functions improve program organization and eliminate code duplication.

Syntax

```
function functionName()

{

      // statements
```

```
}
```

Example

```
function welcome()
{
      document.write("Welcome to ABC College");
}
welcome();
```

The function is executed only when it is called.

Scenario-Based Example

A college website displays the same welcome message on multiple webpages. Instead of writing the message repeatedly, it is placed inside a function and called whenever required.

7.5 Arrow Functions

Definition: An arrow function is a shorter and more concise way of writing a JavaScript function using the arrow (=>) operator.

Arrow functions improve code readability and are widely used in modern JavaScript.

Syntax

```
const functionName = () =>
{
      // statements
}
```

Example

```
const welcome = () =>
{
    document.write("Welcome Students");
};
welcome();
```

The output is the same as a normal function, but the syntax is shorter.

Scenario-Based Example

A company develops a modern web application where developers prefer arrow functions because they are concise and easier to maintain.

7.6 Complete HTML Program

```
<!DOCTYPE html>
<html>
<head>
    <title>JavaScript Functions</title>
</head>
<body>
<button onclick="welcome()">
Click Here
</button>
<script>
function welcome()
{
    document.write("<h2>Welcome to ABC College of Engineering</h2>");
```

```
}
</script>
</body>
</html>
```

7.7 Program Explanation

The webpage contains a button that calls the welcome() function when clicked. The function displays a welcome message using the document.write() method. This example demonstrates how functions help organize reusable code and respond to user actions.

7.8 Scenario-Based Questions

Scenario 1

A college website checks whether a student has secured 50 or more marks before displaying Pass.

Question: Which JavaScript statement should be used?

Answer: if...else

Scenario 2

A university assigns grades A, B, C, and D based on different mark ranges.

Question: Which conditional statement is most suitable?

Answer: else if

Scenario 3

A shopping website displays ten products using the same code repeatedly.

Question: Which JavaScript concept should be used?

Answer: for loop

Scenario 4

A banking application repeatedly displays the transaction menu until the customer exits.

Question: Which loop is suitable?

Answer: while loop

Scenario 5

A college website displays the same welcome message on multiple pages.

Question: Which JavaScript concept helps avoid writing the same code repeatedly?

Answer: Function

Scenario 6

A software company develops a modern JavaScript application and prefers concise function syntax.

Question: Which type of function is suitable?

Answer: Arrow Function

7.9 Important Questions

Question 1: What is a conditional statement?

Answer: A conditional statement is a decision-making statement that executes different blocks of code based on whether a condition is true or false.

Question 2: What is the purpose of a loop?

Answer: A loop repeatedly executes a block of code until a specified condition becomes false.

Question 3: What is a function?

Answer: A function is a reusable block of code designed to perform a specific task whenever it is called.

Question 4: What is an arrow function?

Answer: An arrow function is a shorter syntax for writing JavaScript functions using the => operator.

Question 5: Name the three commonly used loops in JavaScript.

Answer: for, while, and do...while.

7.10 Summary

Conditional statements, loops, and functions are essential programming constructs in JavaScript. Conditional statements enable programs to make decisions based on specified conditions, loops allow repeated execution of code, and functions organize reusable logic into manageable blocks. Arrow functions provide a concise syntax for writing functions in modern JavaScript. These concepts help developers create efficient, interactive, and maintainable web applications. In the next topic, we will study Scope, Arrays, Objects, DOM, and Events, which introduce data organization, webpage manipulation, and user interaction in JavaScript.

8. Scope, Arrays, Objects, DOM and Events

8.1 Introduction

As JavaScript programs become larger, developers need better ways to organize data, control where variables can be used, and respond to user interactions. JavaScript provides several important concepts to achieve these goals. Scope controls the accessibility of variables, arrays store multiple values in a single variable, objects represent real-world entities using properties, the Document Object Model (DOM) allows JavaScript to access and modify HTML elements, and events enable webpages to respond to user actions such as clicking buttons or entering data. Together, these concepts help developers build interactive and well-structured web applications.

8.2 Scope

Definition: Scope is the area of a program where a variable can be accessed or used.

A variable is not always available throughout the entire program. Depending on where it is declared, it may be accessible only within a function or throughout the script. Understanding scope helps prevent errors and improves program organization.

Types of Scope

JavaScript commonly uses two types of scope:

- Global Scope
- Local Scope

1. Global Scope

Definition: A global variable is declared outside all functions and can be accessed from anywhere in the program.

Example

```
let college = "ABC College";

function display()

{

      document.write(college);

}

display();
```

In this example, the variable college is available inside the function because it is declared globally.

Scenario-Based Example

A university website stores the university name in a global variable because it is displayed on every webpage.

2. Local Scope

Definition: A local variable is declared inside a function and can be accessed only within that function.

Example

```
function student()

{

      let name = "Rahul";

      document.write(name);

}

student();
```

The variable name cannot be accessed outside the student() function.

Scenario-Based Example

A banking application stores a temporary transaction number inside a function. After the transaction is completed, the variable is no longer required.

8.3 Arrays

Definition: An array is a data structure that stores multiple values of the same or different data types in a single variable.

Instead of creating separate variables for similar data, an array stores all values together. Each value in an array is identified by an index, which starts from 0.

Syntax

```
let arrayName = [value1, value2, value3];
```

Example

```
let departments = ["CSE", "AI", "IT"];

document.write(departments[0]);
```

Output

CSE

Here, departments[0] refers to the first element in the array.

Scenario-Based Example

A college website stores the names of different departments in an array so they can be displayed on the webpage whenever required.

8.4 Objects

Definition: An object is a collection of related data stored as key-value pairs.

Objects are useful for representing real-world entities such as students, employees, products, or books.

Syntax

```
let objectName =
{
    property1 : value1,
    property2 : value2
};
```

Example

```
let student =
{
    name : "Ananya",
    age : 18,
    department : "CSE"
};
document.write(student.name);
```

Output

Ananya

Scenario-Based Example

A student management system stores each student's name, roll number, and department as properties of a student object.

8.5 Document Object Model (DOM)

Definition: The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree of objects, allowing JavaScript to access, modify, add, or remove HTML elements dynamically.

When a webpage is loaded, the browser creates the DOM. JavaScript uses the DOM to interact with webpage elements without reloading the page.

Common DOM Method

One of the most commonly used DOM methods is:

```
document.getElementById("id");
```

This method selects an HTML element using its id attribute.

Example

```
<p id="message">Welcome Students</p>
<script>
document.getElementById("message").innerHTML = "Welcome to ABC College";
</script>
```

The paragraph text changes without reloading the webpage.

Scenario-Based Example

A college website updates the admission notification dynamically after the administrator changes the information.

8.6 Events

Definition: An event is an action that occurs in the browser and can be detected by JavaScript to execute a specific block of code.

Events make webpages interactive by responding to user actions.

Common Events

Event	Description
onclick	Occurs when an element is clicked
ondblclick	Occurs when an element is double-clicked
onmouseover	Occurs when the mouse pointer moves over an element
onmouseout	Occurs when the mouse pointer leaves an element
onchange	Occurs when the value of an element changes
onkeyup	Occurs when a keyboard key is released

Example

```
<button onclick="showMessage()">
Click Here
</button>
<script>
function showMessage()
{  alert("Admission Open"); }
</script>
```

When the button is clicked, the browser displays the message Admission Open.

Scenario-Based Example

A university website displays important admission information immediately after the visitor clicks the Apply Now button.

8.7 Complete HTML Program

```
<!DOCTYPE html>
<html>
<head>
    <title>DOM and Events</title>
</head>
<body>
<h2 id="heading">Welcome Students</h2>
<button onclick="changeText()">
Click Here
</button>
<script>
function changeText()
{
    document.getElementById("heading").innerHTML =
    "Welcome to ABC College of Engineering";
}
</script>
</body>
</html>
```

8.8 Program Explanation

The webpage contains a heading and a button. The heading is identified using the id attribute. When the user clicks the button, the changeText() function is executed. Inside the function, the document.getElementById() method selects the heading, and the innerHTML property changes its content. This demonstrates how JavaScript uses the DOM and events to update webpage content dynamically without refreshing the page.

8.9 Scenario-Based Questions

Scenario 1

A university website stores the university name so that it can be displayed on every webpage.

Question: Which type of scope is suitable?

Answer: Global Scope.

Scenario 2

A college website stores the names of all departments in a single variable.

Question: Which JavaScript concept should be used?

Answer: Array.

Scenario 3

A student management system stores a student's name, age, and department together.

Question: Which JavaScript concept is most suitable?

Answer: Object.

Scenario 4

A webpage changes the text of a heading after a button is clicked.

Question: Which JavaScript concept allows access to the HTML element?

Answer: Document Object Model (DOM).

Scenario 5

A shopping website displays a confirmation message when the customer clicks the Place Order button.

Question: Which JavaScript event is commonly used?

Answer: onclick.

8.10 Important Questions

Question 1: What is scope in JavaScript?

Answer: Scope is the area of a program where a variable can be accessed or used.

Question 2: What is an array?

Answer: An array is a data structure that stores multiple values in a single variable.

Question 3: What is an object in JavaScript?

Answer: An object is a collection of related data stored as key-value pairs.

Question 4: What is the Document Object Model (DOM)?

Answer: The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree of objects, allowing JavaScript to access and modify webpage elements dynamically.

Question 5: What is an event in JavaScript?

Answer: An event is an action that occurs in the browser and triggers the execution of JavaScript code.

8.11 Summary

Scope, arrays, objects, the Document Object Model (DOM), and events are essential concepts in JavaScript programming. Scope determines where variables can be accessed, arrays store multiple values efficiently, and objects organize related information using key-value pairs. The DOM enables JavaScript to interact with and modify HTML elements dynamically, while events allow webpages to respond to user actions such as mouse clicks and keyboard input. Together, these concepts form the foundation for developing interactive and dynamic web applications.